МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Инженерный бизнес и менеджмент”
Кафедра “Промышленная логистика (Бизнес-информатика)”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1

Выполнил:
Студент группы ИБМ3-34Б

Гараев Р. А.

Преподаватель:

Правдина А. Д.

Цель лабораторной работы: изучение основных конструкций языка Python.

Описание задания

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и ДЕЙСТВИТЕЛЬНЫЕ корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A, B, C могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования.

Листинг программы

1. Процедурная парадигма — Python, файл main.py

```
import sys
import math


def is_valid(value: str) -> bool:
    try:
        return math.isfinite(float(value))
    except (TypeError, ValueError):
        return False


def get_coefficient(args, index, name):
    if len(args) > index and is_valid(args[index]):
        return float(args[index])
    while True:
        val = input(f"Введите коэффициент {name}: ").strip()
        if is_valid(val):
            return float(val)
        print("Ошибка ввода. Повторите попытку.")


def solve_quadratic(a, b, c):
    d = b * b - 4 * a * c
    if d < 0: return []
    if d == 0: return [-b / (2 * a)]
    sqrt_d = math.sqrt(d)
    return [(-b + sqrt_d) / (2 * a), (-b - sqrt_d) / (2 * a)]


def solve_biquadratic(a, b, c):
    if a == 0:
```

```
        if b == 0: return [], c == 0
        y_roots = [-c / b]
    else:
        y_roots = solve_quadratic(a, b, c)

    x_roots = set()
    for y in y_roots:
        if y > 0:
            r = math.sqrt(y)
            x_roots.update((r, -r))
        elif y == 0:
            x_roots.add(0.0)
    return sorted(x_roots), False


def main():
    args = sys.argv[1:]
    print("=== Решение биквадратного уравнения ===\n")

    a = get_coefficient(args, 0, "A")
    b = get_coefficient(args, 1, "B")
    c = get_coefficient(args, 2, "C")

    print(f"\nУравнение: {a}*x^4 + ({b})*x^2 + ({c}) = 0")

    roots, infinite = solve_biquadratic(a, b, c)

    if infinite:
        print("Корень — любое действительное число.")
    elif not roots:
        print("Действительных корней нет.")
    else:
        print("Корни:")
```

```
        for i, x in enumerate(roots, start=1):
            print(f"  x{i} = {x}")



if __name__ == "__main__":
    main()
```

2. Объектно-ориентированная парадигма — Python, файл main_oop.py (дополнительное задание 1)

```
import sys
import math



class CoefficientReader:
    def __init__(self, args):
        self.args = args

    @staticmethod
    def is_valid(value) -> bool:
        try:
            return math.isfinite(float(value))
        except (TypeError, ValueError):
            return False

    def read(self, index, name):
        if len(self.args) > index and self.is_valid(self.args[index]):
            return float(self.args[index])
        while True:
            val = input(f"Введите коэффициент {name}: ").strip()
            if self.is_valid(val):
                return float(val)
            print("Ошибка ввода. Повторите попытку.")



class BiquadraticEquation:
    def __init__(self, a, b, c):
        self.a, self.b, self.c = a, b, c
```

```
        self.infinite = False

    def __str__(self):
        return f"{self.a}*x^4 + ({self.b})*x^2 + ({self.c}) = 0"

    def _roots_y(self):
        if self.a == 0:
            if self.b == 0:
                self.infinite = self.c == 0
                return []
            return [-self.c / self.b]
        d = self.b * self.b - 4 * self.a * self.c
        if d < 0: return []
        if d == 0: return [-self.b / (2 * self.a)]
        sqrt_d = math.sqrt(d)
        return [(-self.b + sqrt_d) / (2 * self.a), (-self.b - sqrt_d) / (2 * self.a)]

    def roots(self):
        self.infinite = False
        x_roots = set()
        for y in self._roots_y():
            if y > 0:
                r = math.sqrt(y)
                x_roots.update((r, -r))
            elif y == 0:
                x_roots.add(0.0)
        return sorted(x_roots)


class BiquadraticApp:
    def __init__(self, args):
        self.reader = CoefficientReader(args)

    def run(self):
        print("=== Решение биквадратного уравнения (ООП) ===\n")
        eq = BiquadraticEquation(
```

```
            self.reader.read(0, "A"),
            self.reader.read(1, "B"),
            self.reader.read(2, "C")
        )

        print(f"\nУравнение: {eq}")
        roots = eq.roots()

        if eq.infinite:
            print("Корень — любое действительное число.")
        elif not roots:
            print("Действительных корней нет.")
        else:
            print("Корни:")
            for i, x in enumerate(roots, start=1):
                print(f"  x{i} = {x}")


if __name__ == "__main__":
    BiquadraticApp(sys.argv[1:]).run()
```

3. Программа на языке C++ — файл cpp/main.cpp (дополнительное задание 2)

```
#include <algorithm>

#include <cmath>

#include <iostream>

#include <string>

#include <vector>


#ifdef _WIN32

#include <windows.h>

#endif
```

```
std::string trim(const std::string &s) {
    size_t begin = s.find_first_not_of(" \t\r\n");
    if (begin == std::string::npos) return "";
    return s.substr(begin, s.find_last_not_of(" \t\r\n") - begin + 1);
}

bool tryParse(const std::string &raw, double &result) {
    std::string s = trim(raw);
    if (s.empty()) return false;
    try {
        size_t pos = 0;
        double value = std::stod(s, &pos);
        if (pos != s.size() || !std::isfinite(value)) return false;
        result = value;
        return true;
    } catch (...) {
        return false;
    }
}

double readFromKeyboard(const std::string &name) {
    std::string line;
    double value;
    while (true) {
        std::cout << "Введите коэффициент " << name << ": ";
```

```
        if (!std::getline(std::cin, line)) std::exit(1);
        if (tryParse(line, value)) return value;
        std::cout << "Ошибка ввода. Повторите попытку.\n";
    }
}

double getCoefficient(int argc, char *argv[], int index, const std::string &name) {
    if (argc > index) {
        double value;
        if (tryParse(argv[index], value)) return value;
    }
    return readFromKeyboard(name);
}

std::vector<double> solveQuadratic(double a, double b, double c) {
    double d = b * b - 4 * a * c;
    if (d < 0) return {};
    if (d == 0) return {-b / (2 * a)};
    double sqrtD = std::sqrt(d);
    return {(-b + sqrtD) / (2 * a), (-b - sqrtD) / (2 * a)};
}

std::vector<double> solveBiquadratic(double a, double b, double c, bool &infinite) {
    infinite = false;
    std::vector<double> yRoots;
```

```
    if (a == 0) {
        if (b == 0) {
            infinite = (c == 0);
            return {};
        }
        yRoots.push_back(-c / b);
    } else {
        yRoots = solveQuadratic(a, b, c);
    }

    std::vector<double> xRoots;
    for (double y : yRoots) {
        if (y > 0) {
            double r = std::sqrt(y);
            xRoots.push_back(r);
            xRoots.push_back(-r);
        } else if (y == 0) {
            xRoots.push_back(0.0);
        }
    }
    std::sort(xRoots.begin(), xRoots.end());
    xRoots.erase(std::unique(xRoots.begin(), xRoots.end()), xRoots.end());
    return xRoots;
}
```

```
int main(int argc, char *argv[]) {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
    std::cout << "=== Решение биквадратного уравнения ===\n\n";

    double a = getCoefficient(argc, argv, 1, "A");
    double b = getCoefficient(argc, argv, 2, "B");
    double c = getCoefficient(argc, argv, 3, "C");

    std::cout << "\nУравнение: " << a << "*x^4 + (" << b << ")*x^2 + (" << c << ") = 0\n";

    bool infinite;
    std::vector<double> roots = solveBiquadratic(a, b, c, infinite);

    if (infinite) {
        std::cout << "Корень — любое действительное число.\n";
    } else if (roots.empty()) {
        std::cout << "Действительных корней нет.\n";
    } else {
        std::cout << "Корни:\n";
        for (size_t i = 0; i < roots.size(); ++i) {
            std::cout << "  x" << i + 1 << " = " << roots[i] << "\n";
        }
```

```
    }

    return 0;

}
```

Экранные формы с примерами выполнения программы

1. Процедурная программа на Python (main.py)

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 ===

Введите коэффициент A: 0
Введите коэффициент B: 1
Введите коэффициент C: -4

Уравнение: 0.0*x^4 + (1.0)*x^2 + (-4.0) = 0
A = 0: уравнение не биквадратное, решается как B*x^2 + C = 0.
Действительные корни:
  x1 = -2.0
  x2 = 2.0

Process finished with exit code 0
```

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 ===

Введите коэффициент A: 0
Введите коэффициент B: 1
Введите коэффициент C: 9

Уравнение: 0.0*x^4 + (1.0)*x^2 + (9.0) = 0
A = 0: уравнение не биквадратное, решается как B*x^2 + C = 0.
Действительных корней нет.

Process finished with exit code 0
```

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 ===

Введите коэффициент A: 1
Введите коэффициент B: -5
Введите коэффициент C: 4

Уравнение: 1.0*x^4 + (-5.0)*x^2 + (4.0) = 0
Дискриминант (для замены y = x^2): D = 9.0
Действительные корни:
  x1 = -2.0
  x2 = -1.0
  x3 = 1.0
  x4 = 2.0

Process finished with exit code 0
```

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 ===

Введите коэффициент A: 1
Введите коэффициент B: -4
Введите коэффициент C: 0

Уравнение: 1.0*x^4 + (-4.0)*x^2 + (0.0) = 0
Дискриминант (для замены y = x^2): D = 16.0
Действительные корни:
  x1 = -2.0
  x2 = 0.0
  x3 = 2.0

Process finished with exit code 0
```

2. Объектно-ориентированная программа на Python (main_oop.py)

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 (ООП) ===

Введите коэффициент A: 2
Введите коэффициент B: 8
Введите коэффициент C: 0

Уравнение: 2.0*x^4 + (8.0)*x^2 + (0.0) = 0
Дискриминант (для замены y = x^2): D = 64.0
Действительные корни:
  x1 = 0.0
```

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 (ООП) ===

Введите коэффициент A: 4
Введите коэффициент B: 1
Введите коэффициент C: 1

Уравнение: 4.0*x^4 + (1.0)*x^2 + (1.0) = 0
Дискриминант (для замены y = x^2): D = -15.0
Действительных корней нет.
```

3. Программа на C++ (cpp/main.cpp)

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 (C++) ===

Введите коэффициент A: 3
Введите коэффициент B: 2
Введите коэффициент C: 1

Уравнение: 3*x^4 + (2)*x^2 + (1) = 0
Дискриминант (для замены y = x^2): D = -8
Действительных корней нет.
```

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 (C++) ===

Введите коэффициент A: 1
Введите коэффициент B: -10
Введите коэффициент C: 9

Уравнение: 1*x^4 + (-10)*x^2 + (9) = 0
Дискриминант (для замены y = x^2): D = 64
Действительные корни:
  x1 = -3
  x2 = -1
  x3 = 1
  x4 = 3
```

```
=== Решение биквадратного уравнения A*x^4 + B*x^2 + C = 0 (C++) ===

Введите коэффициент A: 1
Введите коэффициент B: -1
Введите коэффициент C: -6

Уравнение: 1*x^4 + (-1)*x^2 + (-6) = 0
Дискриминант (для замены y = x^2): D = 25
Действительные корни:
  x1 = -1.73205
  x2 = 1.73205
```